

Lecture 3 Notes: Maven Deep Dive

Topic: JAR Files, Maven, `pom.xml`, Dependencies, Repositories, and Maven Lifecycle

Goal: Understand why Maven exists and how it helps Java projects manage structure, dependencies, builds, and packaging.

1. The Basic Journey of Java Code

Before understanding Maven, we first need to understand the basic flow of a Java program.

```
Hello.java -- compiled by javac --> Hello.class -- run by JVM --> output
```

At the core Java level:

```
.java file    = source code written by the developer  
.class file   = compiled bytecode understood by the JVM
```

So, Java code is not directly executed as `.java` files. First, it is compiled into `.class` files, and then the JVM runs those `.class` files.

2. What Happens in a Real Java Project?

A real project usually does not have only one Java class.

It may have many files like:

```
User.java  
Order.java  
Payment.java  
EmailService.java
```

```
InvoiceGenerator.java
Main.java
```

After compilation, each Java file becomes a `.class` file:

```
User.class
Order.class
Payment.class
EmailService.class
InvoiceGenerator.class
Main.class
```

Now imagine sharing this project with another developer.

Sending many separate `.class` files is not practical because:

- Files can get misplaced.
- Package/folder structure can break.
- Some files may be missed.
- Resource files may not be included.
- Running the project becomes confusing.

Java needed a standard way to bundle compiled code and related files together.

That standard bundle is called a **JAR file**.

3. What is a JAR File?

JAR stands for **Java Archive**.

A JAR file is basically a packaged bundle for Java projects. You can think of it like a ZIP file designed specifically for Java.

A JAR file can contain:

```
.class files
folders/packages
images
properties files
```

```
configuration files
metadata
```

In simple words:

A JAR file is a standard way to package Java compiled code and related resources into one file.

4. Why Do We Need a JAR File?

There are two major reasons why JAR files are important.

Reason 1: To Share Java Code Easily

Suppose you create a useful calculator library.

Instead of sending all compiled `.class` files separately, you can package them into:

```
calculator.jar
```

Now another developer can add this JAR to their project and use your compiled code.

This is how many Java libraries are shared.

5. Library vs Application

Before moving further, it is important to understand the difference between a **library** and an **application**.

Term	Meaning
Library	Code created for other developers to use inside their projects. It usually does not run independently.
Application	Code created for the end user. It can run as a complete program, usually with a <code>main()</code> method or an application entry point.

Example:

calculator.jar

If this JAR only provides reusable calculator functions, it is a **library JAR**.

But if it contains a complete runnable program, it is an **application JAR**.

6. Reason 2: To Use External Libraries

When we use libraries like:

```
Spring Core  
MySQL Connector  
Hibernate  
Jackson
```

we are usually using someone else's JAR files.

For example:

When we say, "I am using Spring Core", practically our project is using Spring Core JAR files.

Similarly:

When we say, "I am using MySQL driver", our project is using the MySQL driver JAR.

So, a library is often just compiled Java code packaged as a JAR file.

7. Important Note: Library JAR vs Spring Boot Executable JAR

A normal library JAR usually does **not** contain all of its dependency JARs inside it.

For example, if you create a reusable library, its JAR may contain your compiled classes, but its dependency information is usually managed separately through Maven using `pom.xml`.

However, a Spring Boot application is commonly packaged as an **executable JAR**. In that case, Spring Boot can package the application code along with required dependencies so that the application can run as one complete executable JAR.

So:

Normal library JAR

→ Usually contains your compiled code, not all external dependency JARs inside it.

Spring Boot executable JAR

→ Can contain application code + required dependencies, making it runnable as one JAR.

This is why we often call Spring Boot output a **fat JAR** or **executable JAR**.

8. External Libraries and Classpath

Suppose your Java code uses an external JAR.

Conceptually, the flow looks like this:

```
Your Java code
+
External JAR file
↓
Compiled together
↓
Run by JVM
```

During compile time and runtime, Java must know where to find the required classes.

For example, Java may need to know:

“This class is not in my project. It is inside this external JAR.”

This connection is handled using something called the **classpath**.

9. What is Classpath?

The **classpath** is the place where Java searches for classes.

Java searches for classes in:

- Your own project classes
- External JAR files
- Other configured class locations

If the class is present in your project, Java can find it there.

If the class is inside an external JAR, Java must know the location of that JAR.

In simple words:

| Classpath tells Java where to search for required classes.

10. Where Maven Comes Into the Picture

If a project needs only one external JAR, we can manually download it and add it to the project.

But real projects usually need many third-party libraries.

Then many problems appear:

- Where should we download these JARs from?
- Which version should we use?
- What if one JAR depends on another JAR?
- What if two teammates use different versions?
- What if we forget to add one required JAR?
- What if version 1 works, but version 2 breaks compatibility?

This is exactly where **Maven** becomes useful.

Maven says:

| Do not manually download and manage JAR files. Tell me what dependency your project needs, and I will download and manage the required JARs for you.

11. What is Maven?

Maven is a project management and build automation tool for Java projects.

It is important to understand that Maven does not exist only because of Spring.

Maven is independent of Spring.

You can use Maven with:

- Core Java projects
- Spring projects
- Spring Boot projects
- Hibernate projects
- Many other Java-based projects

In simple words:

Maven helps us manage project structure, dependencies, builds, packaging, and project configuration in a standard way.

12. What Maven Does Practically

Maven mainly helps with these things:

1. It gives a standard project structure.
2. It compiles Java code.
3. It runs tests.
4. It creates JAR/WAR files.
5. It downloads external JAR libraries.
6. It manages dependency versions.
7. It uses plugins to perform build-related tasks.

This makes Java projects more consistent and easier to work with, especially in teams.

13. Maven and IDEs

Maven can be used from the command line.

For example:

```
mvn clean install
```

But most modern IDEs also support Maven directly, such as:

- IntelliJ IDEA
- Eclipse
- VS Code

Every IDE may have its own project structure, but Maven gives us a standard structure that works across tools.

That is why Maven is helpful in real projects.

14. Convention Over Configuration

Maven follows a principle called:

| Convention over Configuration

This means Maven already assumes a standard project structure.

If we follow that structure, we do not need to manually configure everything.

For example, Maven expects main Java code inside:

```
src/main/java
```

and test code inside:

```
src/test/java
```

Because of this standard convention, Maven automatically knows where to find source code, test code, resources, and where to place generated output.

15. Standard Maven Project Structure

A typical Maven project looks like this:

```
my-maven-project/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   └── resources/
│   └── test/
│       ├── java/
│       └── resources/
└── target/
```

Now let us understand each part.

pom.xml

This file contains information about the complete Maven project.

It tells Maven:

- Project name
- Project version
- Required dependencies
- Plugins
- Build configuration
- Parent configuration, if any

src/main/java

This folder contains the main Java source code of the application.

Example:

Controller classes
Service classes
Repository classes
Utility classes
Main application class

src/main/resources

This folder contains non-Java files needed by the main application.

Example:

application.properties
application.yml
configuration files
static files
templates

src/test/java

This folder contains test code.

Example:

unit tests
integration tests

src/test/resources

This folder contains resources needed only during testing.

Example:

test configuration files
test data files

target

The `target` folder is created by Maven during the build process.

It can contain:

```
compiled .class files
test reports
generated sources
final JAR/WAR file
temporary build files
```

If we delete the `target` folder, Maven can recreate it during the next build.

So, `target` is generated output, not source code.

16. The Most Important File in Maven: `pom.xml`

Every Maven project has a file called:

```
pom.xml
```

POM stands for **Project Object Model**.

The `pom.xml` file is the heart of a Maven project.

It tells Maven:

- What the project is
- Which version the project has
- What type of output should be created
- Which external libraries are needed
- Which plugins should be used
- Whether there is any parent configuration

17. Basic Structure of `pom.xml`

Every `pom.xml` starts with a root `<project>` tag.

A simple POM may look like this:

Coder Army

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                                https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.coderarmy</groupId>
    <artifactId>calculator-app</artifactId>
    <version>1.0.0</version>

</project>
```

The extra `xmlns` lines define the XML namespace and structure.

For beginners, we do not need to deeply focus on them at this stage.

18. `<modelVersion>`

```
<modelVersion>4.0.0</modelVersion>
```

This tells Maven which POM model version this file follows.

Most Maven projects use:

```
4.0.0
```

19. Maven Coordinates

These three tags are very important:

```
<groupId>com.coderarmy</groupId>
<artifactId>calculator-app</artifactId>
```

```
<version>1.0.0</version>
```

Together, they are called **Maven coordinates**.

Maven uses these coordinates to uniquely identify a project or dependency.

<groupId>

`groupId` usually represents the organization, company, or domain that owns the project.

Example:

```
<groupId>com.coderarmy</groupId>
```

<artifactId>

`artifactId` is the name of the project or module.

Example:

```
<artifactId>calculator-app</artifactId>
```

<version>

`version` tells which version of the project is being built.

Example:

```
<version>1.0.0</version>
```

20. What is **SNAPSHOT** in Maven?

Sometimes we see versions like:

```
<version>1.0.0-SNAPSHOT</version>
```

SNAPSHOT means the project is still under development.

It is not considered a final stable release yet.

In simple words:

```
1.0.0          → stable release version
1.0.0-SNAPSHOT → development version
```

21. `<packaging>` Tag

The `<packaging>` tag tells Maven what type of output should be created.

Example:

```
<packaging>jar</packaging>
```

Common packaging types:

Packaging	Meaning
<code>jar</code>	Creates a JAR file
<code>war</code>	Creates a WAR file, usually for traditional web applications
<code>pom</code>	Used for parent or aggregator projects

If we do not mention packaging, Maven defaults to:

```
jar
```

22. `<properties>` Tag

The `<properties>` tag is used to define reusable values in the POM.

Example:

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.source
```

```
Encoding>
</properties>
```

Here, we are telling Maven:

- Use Java 17 source code.
- Compile it for Java 17.
- Use UTF-8 encoding.

Properties make the POM cleaner because the same value can be reused in multiple places.

23. `<dependencies>` Tag

The `<dependencies>` section contains external libraries required by the project.

Example:

```
<dependencies>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.x.x</version>
  </dependency>
</dependencies>
```

Each dependency is also identified using Maven coordinates:

```
groupId + artifactId + version
```

When Maven reads this dependency, it downloads the required JAR file from a Maven repository.

24. Dependency Example: Hibernate

A Hibernate dependency may look like this:

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.x.x</version>
</dependency>
```

This tells Maven that the project needs Hibernate Core.

Maven will download the Hibernate JAR and also manage its required supporting dependencies.

25. Dependency Example: Spring Boot Starter

In Spring Boot projects, we commonly use starter dependencies.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A starter dependency does not mean only one JAR.

It usually brings a group of related dependencies needed for a particular feature.

For example, `spring-boot-starter-web` brings dependencies related to web application development.

26. Transitive Dependencies

Sometimes one dependency needs other dependencies to work.

For example:

```
Your project needs A
A needs B
B needs C
```

In Maven, if you add dependency A, Maven can automatically download B and C also.

These are called **transitive dependencies**.

In simple words:

| Transitive dependencies are dependencies of your dependencies.

This is one of the biggest reasons Maven is useful.

Without Maven, we would have to manually find and download every required JAR.

27. Maven Plugins

Dependencies are libraries that our project uses.

Plugins are tools that Maven uses to perform tasks.

Maven plugins can help with:

- Compiling code
- Running tests
- Creating JAR files
- Creating reports
- Running a Spring Boot application
- Packaging the application

For example, the Spring Boot Maven plugin helps package and run Spring Boot applications.

In simple words:

Dependency → used by our application code
Plugin → used by Maven during the build process

28. Where Does Maven Download JAR Files From?

Maven downloads dependencies from Maven repositories.

A **Maven repository** is a storage place where Maven artifacts are kept.

Artifacts can include:

```
JAR files
WAR files
POM files
plugins
other build artifacts
```

29. Types of Maven Repositories

At a beginner level, remember these three types:

1. Local repository
2. Maven Central repository
3. Remote repository

30. Local Repository

The local repository is a folder on your own computer.

Default location:

```
Mac/Linux:
~/.m2/repository

Windows:
C:\Users\<your-username>\.m2\repository
```

The local repository has two main jobs:

1. It caches downloaded dependencies.
2. It stores your own locally installed artifacts.

When Maven downloads a dependency for the first time, it stores it in the local repository.

Because of this, the first build may be slower, but later builds are faster because dependencies are already cached.

31. What Happens During `mvn install` ?

When we run:

```
mvn install
```

Maven builds the project and stores the final artifact in the local Maven repository.

Example location:

```
~/.m2/repository/com/example/MavenDemo/1.0/
```

This allows other local Maven projects on the same machine to use that artifact.

32. Maven Central Repository

Maven Central is a huge public repository where many open-source Java libraries are published.

Libraries like these are commonly available through Maven Central:

```
Spring  
Hibernate  
Jackson  
JUnit  
MySQL Connector
```

When we add a public dependency in `pom.xml`, Maven usually downloads it from Maven Central if it is not already available locally.

33. Remote Repository

A remote repository is any Maven repository that is not on your local machine.

It may be available through:

- The internet
- A company network
- A private repository server

Examples:

```
Maven Central  
Company Nexus repository  
Company Artifactory repository  
GitHub Packages
```

34. Why Companies Use Private Repositories

In companies, not every JAR is public.

A company may create internal libraries that should only be used inside the organization.

For such cases, companies use private Maven repositories like:

```
Nexus  
Artifactory  
GitHub Packages
```

A private repository can be configured in `pom.xml` like this:

```
<repositories>  
  <repository>  
    <id>company - repo</id>  
    <url>https://repo.company.com/maven2</url>  
  </repository>  
</repositories>
```

35. How Maven Searches for a Dependency

Suppose our `pom.xml` has this dependency:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.17.0</version>
</dependency>
```

Maven roughly follows this process:

1. Read pom.xml
2. See that a dependency is required
3. Check the local repository first
4. If found locally, use it
5. If not found locally, check remote repositories
6. Download the JAR and POM
7. Store them in the local repository
8. Use them in the project build

That is why the first build is usually slower than later builds.

The first time, Maven downloads dependencies.

After that, Maven uses the cached dependencies from the local `.m2` repository.

36. What is `settings.xml` ?

`settings.xml` is Maven's user-level configuration file.

It is usually found inside the `.m2` folder.

Common location:

```
~/.m2/settings.xml
```

It can contain user-specific Maven settings such as:

- Repository credentials
- Proxy settings
- Mirror configuration

- Custom local repository location

For beginners, it is enough to remember:

`pom.xml` is project-level configuration, while `settings.xml` is user/machine-level Maven configuration.

37. Why Deleting `.m2/repository` Sometimes Fixes Dependency Errors

Sometimes dependency files inside the local Maven repository may become corrupted or incomplete.

For example:

- Download failed in between
- Internet disconnected during dependency download
- Partial JAR file got saved
- Metadata became inconsistent

In such cases, deleting the problematic dependency folder from `.m2/repository` forces Maven to download it again.

That is why deleting `.m2/repository` or a specific dependency folder sometimes fixes Maven dependency errors.

However, deleting the entire `.m2/repository` means Maven will need to download all dependencies again.

38. Maven Lifecycle

A Java project has a build journey.

For example:

```
Check project → Compile code → Run tests → Create JAR → Store/Deploy artifact
```

Maven calls this ordered build journey a **lifecycle**.

In simple words:

A Maven lifecycle is a standard sequence of steps Maven follows to build and manage a project.

39. The Most Important Rule of Maven Lifecycle

The heart of Maven lifecycle is this rule:

When we run a Maven phase, Maven automatically runs all earlier phases of that lifecycle before it.

For example, if we run:

```
mvn package
```

Maven does not only run `package`.

It runs the earlier required phases first, such as:

```
validate → compile → test → package
```

Similarly, if we run:

```
mvn install
```

Maven runs:

```
validate → compile → test → package → verify → install
```

40. The Three Main Maven Lifecycles

Maven has three main lifecycles:

1. Clean Lifecycle
2. Default Lifecycle

3. Site Lifecycle

At beginner level, we can understand them like this:

Lifecycle	Purpose
Clean	Cleans old build output
Default	Builds, tests, packages, installs, and deploys the project
Site	Generates project documentation and reports

41. Clean Lifecycle

The clean lifecycle is used to remove old build output.

Common command:

```
mvn clean
```

Usually, this deletes the `target` folder.

This is useful when we want a fresh build.

42. Default Lifecycle

The default lifecycle is the main build lifecycle.

Important phases:

```
validate → compile → test → package → verify → install → deploy
```

Each phase represents one step in the build journey.

43. Important Default Lifecycle Phases

Phase	Meaning
<code>validate</code>	Checks whether the project is valid
<code>compile</code>	Compiles <code>.java</code> files into <code>.class</code> files

Phase	Meaning
<code>test</code>	Runs test cases
<code>package</code>	Creates JAR/WAR file
<code>verify</code>	Performs additional checks on the packaged output
<code>install</code>	Stores artifact in local <code>.m2</code> repository
<code>deploy</code>	Uploads artifact to a remote Maven repository

44. `validate` Phase

The `validate` phase checks whether the project is structurally valid.

It checks things like:

```
Is pom.xml present?
Is the POM readable?
Are required project details available?
Is the project structure valid enough to continue?
```

Command:

```
mvn validate
```

45. `compile` Phase

The `compile` phase compiles the main Java code.

Command:

```
mvn compile
```

Flow:

```
src/main/java → target/classes
```

This means Java source files are compiled into `.class` files and placed inside `target/classes`.

46. `test` Phase

The `test` phase runs the test cases of the project.

Command:

```
mvn test
```

Maven compiles and runs tests from:

```
src/test/java
```

This helps verify that the code works as expected before packaging.

47. `package` Phase

The `package` phase creates the final packaged output.

Command:

```
mvn package
```

For a normal Java project, this usually creates a JAR file inside the `target` folder.

Example:

```
target/MavenDemo-1.0.jar
```

For web applications, it may create a WAR file depending on the packaging type.

48. `verify` Phase

The `verify` phase checks whether the packaged output is valid.

Command:

```
mvn verify
```

In bigger projects, this phase can include:

```
integration tests  
code quality checks  
coverage checks  
extra validation
```

For beginners, remember:

`verify` is used to perform additional checks before installing or deploying the artifact.

49. `install` Phase

The `install` phase builds the project and stores the final artifact in the local Maven repository.

Command:

```
mvn install
```

The artifact is stored in:

```
~/.m2/repository
```

This is useful when another local project needs to use this project as a dependency.

50. `deploy` Phase

The `deploy` phase builds the project and uploads the final artifact to a remote Maven repository.

Command:

```
mvn deploy
```

Remote repositories can include:

```
company Nexus  
company Artifactory  
GitHub Packages  
remote Maven repository
```

This is different from pushing code to GitHub.

GitHub usually stores source code.

A Maven remote repository stores built artifacts like JAR files so that other projects can use them as dependencies.

51. Site Lifecycle

The site lifecycle is used to generate project documentation and reports.

Command:

```
mvn site
```

Output usually goes inside:

```
target/site
```

This is separate from the normal build flow.

52. What Happens When We Run `mvn clean install` ?

When we run:

```
mvn clean install
```

Maven runs two lifecycle commands together:

```
clean + install
```

First, Maven runs the clean lifecycle and deletes old build output, usually the `target` folder.

Then, Maven runs the default lifecycle up to the `install` phase:

```
validate → compile → test → package → verify → install
```

So the complete idea is:

```
mvn clean install
= clean old output
+ build the project again
+ run tests
+ package the project
+ install the artifact into local .m2 repository
```

53. Maven Archetypes

A Maven archetype is a project template.

It provides a ready-made structure for creating a new project.

In simple words:

A Maven archetype is a starter template with predefined folder structure and configuration.

It can help generate projects quickly instead of creating everything manually.

Example idea:

```
Template → project structure + basic configuration + boilerplate files
```

Modern Spring Boot projects are usually created using Spring Initializr, but Maven archetypes are still useful to understand as a Maven concept.

Final Revision Summary

In this lecture, we understood the journey from basic Java compilation to Maven project management.

Key points:

- Java source code is compiled into `.class` files.
- A JAR file packages compiled Java code and resources.
- External libraries are usually available as JAR files.
- Java uses classpath to find project classes and external classes.
- Manually managing many JAR files is difficult.
- Maven solves this by managing project structure, dependencies, builds, and packaging.
- `pom.xml` is the heart of a Maven project.
- Maven dependencies are identified using `groupId`, `artifactId`, and `version`.
- Maven downloads dependencies from repositories.
- The local repository is stored inside `.m2/repository`.
- Maven Central is a public repository for open-source Java libraries.
- Remote repositories can be public or private.
- Maven lifecycle defines the standard build flow.
- Running a Maven phase also runs all earlier phases in that lifecycle.
- `mvn clean install` cleans old output and builds the project up to the install phase.
- Maven archetypes are templates for creating projects quickly.

One-Line Takeaway

Maven exists because real Java projects need a standard way to manage structure, dependencies, builds, packaging, and sharing of artifacts.

Coder Army